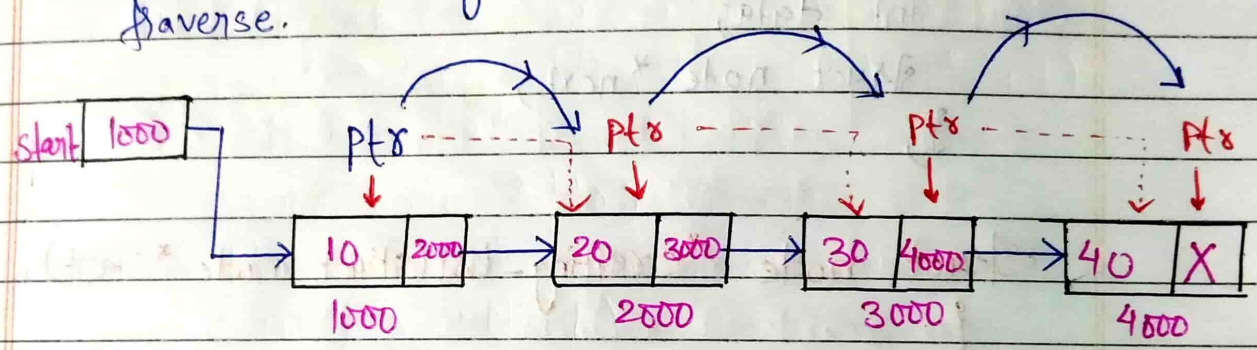
→ Traversing a Linked List :-

Traversing means Accessing each record exactly once. (This accessing is sometimes called "visiting" the record.)

⇒ Algorithm →

Our list is already created, just we have to traverse.



→ It is also used to used to display List.

Algorithm →

Step 1. IF (start == NULL)
print "empty List";
goto step 4

Step 2. Set ptr = start

Step 3. Repeat until ptr != NULL
3.1. ptr = ptr → next

Step 4. Exit

Akash

```
// C program for traversing a linked list
// same as creating & display a linked list,
// adding traversing

#include <stdio.h>
#include <stdlib.h>
struct node
  {
    int data;
    struct node *next;
  };

struct node *creating_list(struct node *start)
  {
    // same as creating a list
  }

struct node *display_list (struct node *start)
  {
    //same as display a list
  }

struct node *traversing_list(struct node *start)
  {
    struct node *ptr;
    if(start==NULL)
      {
        printf("empty list");
      }
```

Akash

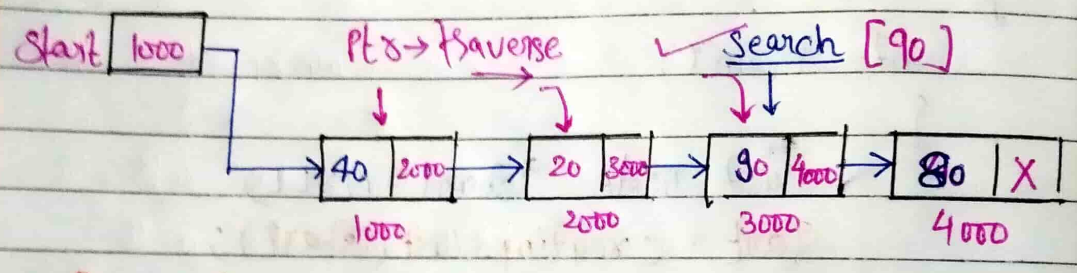
```
else ~~ptr=start~~;
~~else~~ else ~~ptr=start~~
{       ptr=start;
    while (ptr != NULL)      // traversing start
    {
        ptr=ptr->next;
    }
    printf("\n traversing completed");
}
}

int main()
{
    struct node *start=NULL;
    start = creating_list(start);
    start = display_list(start);
    start = traversing_list(start);
    return 0;
}
```

Akash

→ Searching a Linked List :-

Searching a Linked List refers to the process of traversing the list to find a specific value in one of the nodes.

① List is unsorted;



Our List is already created, just we have to search a element (unsorted).

Algorithm

Step 1:
IF start == NULL
print "Empty List"

Step 2: set ptr = start

Step 3: set loc = 1

Step 4: Repeat until ptr != NULL
4.1. if ptr→data == search
print "search element found"
goto step 7

Step 5: Else
5.1. ptr = ptr→next
loc ++;

Akash

Step 6. print "search element not found"

Step 7. exit.

→ Pseudo code for Unsorted List;

```
struct node *ptr;
int search; // search element
int loc=1; // for location or position of element.
printf("\n enter value to search ");
scanf("%d", &search);
if(start==NULL)
  {
    printf("empty list");
  }
else
  {
      ptr=start;
      while(ptr!=NULL)
        {
          if(ptr->data==search)
            {
              printf("\n search %d found at
                position %d", search, loc);
              exit(0);
            }
          else
            {
```

```
        ptr = ptr->next;
        loc++;
    }
  }
}
printf("\n search element not found");
```

// c program for unsorted list search

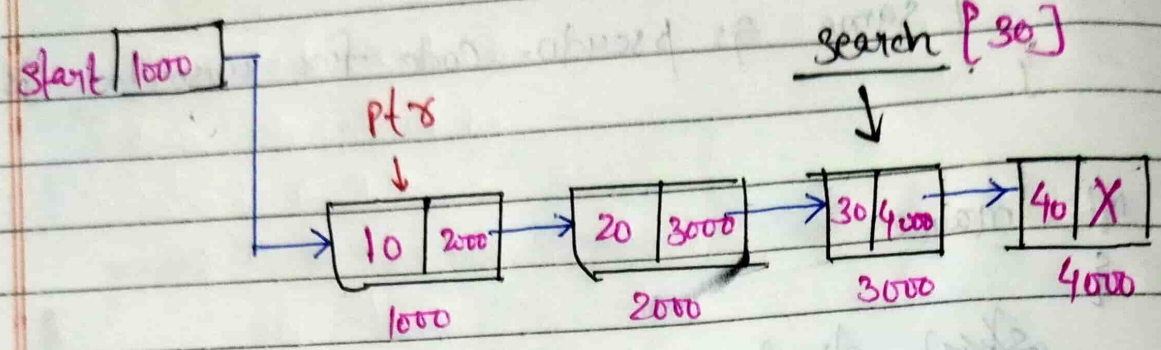
```
#include <stdio.h>
#include <stdlib.h>
struct node
  {
    int data;
    struct node *next;
  };

struct node *create_list(struct node *start)
  {
    // same as creating list
  }

struct node *display_list(struct node *start)
  {
    // same as display list
  }
```

Akash

```
struct node *searching_unsorted_list (struct node *start)
{
    // same as pseudo code for unsorted list;
}

int main()
{
    ~~start~~ struct node *start = NULL;
    start = create_list (start);
    start = display_list (start);
    start = searching_unsorted_list (start);
    return 0;
}
```

Akash

② List is sorted



Our list is already created, we just have to search a element (sorted).

Algorithm →

Step 1. IF start = = NULL
print "empty list";
2. set ptr = start
3. repeat until ptr != NULL
3.1. if(ptr→data < search)
3.1.1. ptr = ptr →next
3.1.2. loc++
4. else if (ptr→data == search)
print "search element found"
goto step 6.

5. printf("search element not found

6. exit

Akash

Pseudo code for sorted list;

```
struct node *ptr;
int search;
int loc=1;
printf("\n enter value to search");
scanf("%d", &search);
if(start==NULL)
  {
    printf("empty list");
  }
else
  {
    ptr=start;
    while(ptr!=NULL)
      {
        if( ptr->data < search)
          {
            ptr=ptr->next;
            loc++;
          }
        else if( ptr->data==search)
          {
            printf("search element found %d
                    at %d position", search, loc);
            exit (0);
          }
      }
  }
printf("search element not found");
```

// C program for sorted list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
};
struct node *create_list(struct node *start)
{
    // same as create_list
}

struct node *display_list (struct node *start)
{
    // same as display_list
}

struct node *searching_sorted_list (struct node *start)
{
    // same as pseudo code for sorted list
}

int main()
{
    struct node *start=NULL;
    start= create_list(start);
    start= display_list(start);
```

Akash

```
start = searching_sorted_list (start);
return 0;
}
```

Akash

Akash